

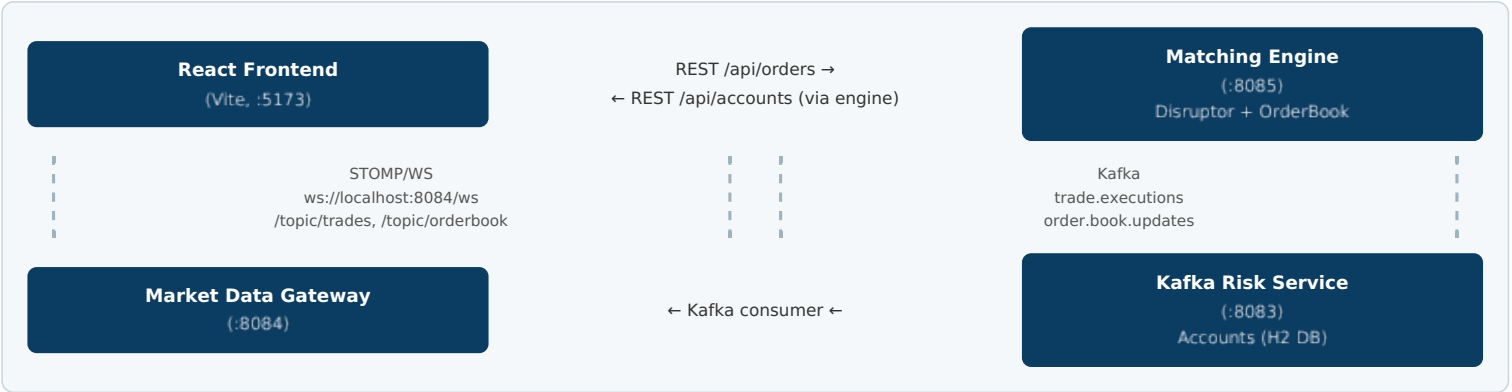
OrderFlow Trading Engine

Technical Documentation

Repository: github.com/kadam-tejesh/orderflow-trading-engine
Generated: August 19, 2026

OrderFlow Trading Engine

A simulated real-time trading platform built as three independent Spring Boot microservices plus a React frontend. It accepts limit/market orders over REST, matches them against an in-memory order book, runs a risk (funds) check before acceptance, and streams trade executions and order book depth to connected clients over WebSocket.



Contents

- [Overview](#)
- [Services](#)
- [Order lifecycle](#)
- [Tech stack](#)
- [Project structure](#)
- [Prerequisites](#)
- [Running locally](#)
- [API reference](#)
- [WebSocket / STOMP topics](#)
- [Kafka topics](#)
- [Configuration](#)
- [Testing](#)
- [Known limitations](#)

Overview

OrderFlow simulates the core pieces of an exchange:

1. A trader submits an order from the web terminal.
2. The **Matching Engine** checks the trader's account has sufficient funds (via a synchronous call to the **Risk Service**), then queues the order on an LMAX Disruptor ring buffer for single-threaded, low-latency matching against an in-memory order book.
3. Any resulting trade is published to Kafka (`trade.executions`); the current top-of-book depth is broadcast to Kafka (`order.book.updates`) on a fixed 100ms schedule.
4. The **Kafka Risk Service** consumes trade executions to log clearing/settlement activity and exposes account balance endpoints used for the pre-trade risk check.
5. The **Market Data Gateway** consumes both Kafka topics and re-publishes them over STOMP/WebSocket so the frontend can render live trades and order book depth without polling.

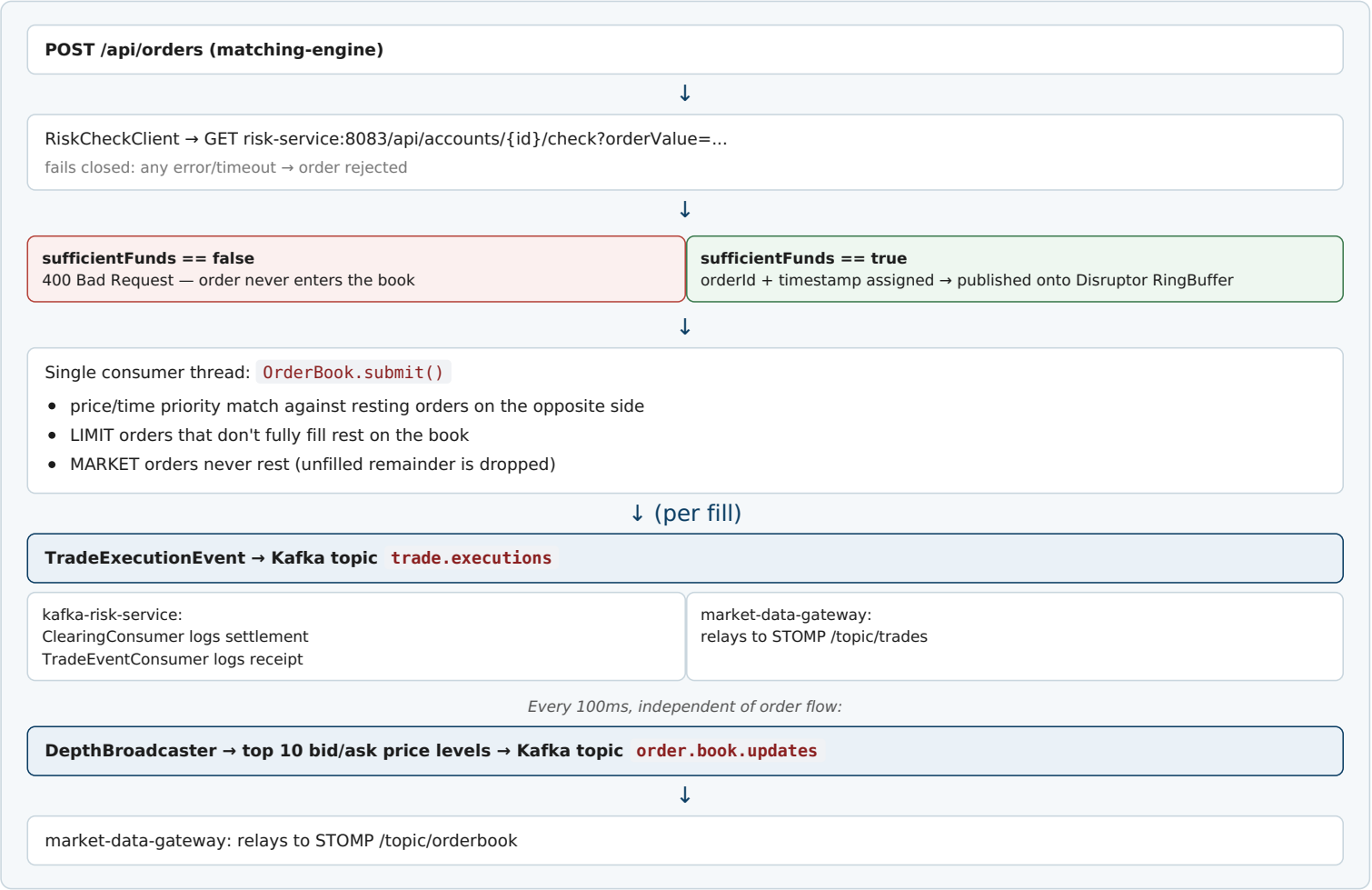
All three backend services are independent Spring Boot applications (there is no parent/multi-module `pom.xml` — each has its own `pom.xml` , `mvnw` , and lifecycle) that communicate via REST and Kafka.

Services

Service	Port	Responsibility	Persistence
matching-engine	8085	Accepts orders, runs the risk check, matches orders via an LMAX Disruptor + price/time-priority order book, emits trades and depth snapshots	In-memory order book (not persisted)
kafka-risk-service	8083	Owens account balances, exposes the funds-check endpoint, consumes trade executions for clearing/logging	H2 in-memory DB (<code>riskdb</code>), console at <code>/h2-console</code>
market-data-gateway	8084	Bridges Kafka topics to browser clients over STOMP/WebSocket (SockJS)	None (stateless relay)

frontend	5173 (Vite dev)	Trading terminal UI: order entry, live order book, trade tape, ticker	None
----------	-----------------------	---	------

Order lifecycle



Tech stack

- **Backend:** Java 21, Spring Boot 4.1.0, Spring Web MVC, Spring for Apache Kafka, Spring Data JPA, Lombok
- **Matching core:** LMAX Disruptor 4.0.0 (single-producer ring buffer, `BlockingWaitStrategy`)
- **Messaging:** Apache Kafka (topics: `trade.executions`, `order.book.updates`)
- **Risk store:** H2 in-memory database
- **Real-time transport:** Spring WebSocket (STOMP over SockJS)
- **Frontend:** React 19, Vite, `@stomp/stompjs`, `sockjs-client`

Project structure

```
orderflow-trading-engine/
├── backend/
│   ├── matching-engine/           # order intake, matching, trade/depth publishing
│   │   ├── src/main/java/com/orderflow/matching_engine/
│   │   │   ├── controller/        # OrderController (REST)
│   │   │   ├── disruptor/         # DisruptorConfig, OrderEvent, OrderPublisher
│   │   │   ├── order/             # Order, OrderBook (matching logic)
│   │   │   ├── event/             # TradeExecutionEvent
│   │   │   ├── gateway/           # DepthBroadcaster (scheduled Kafka publish)
│   │   │   └── risk/              # RiskCheckClient (calls kafka-risk-service)
│   ├── kafka-risk-service/        # accounts, funds check, clearing log
│   │   ├── src/main/java/com/orderflow/kafka_risk_service/
│   │   │   ├── risk/              # Account, AccountRepository, RiskService, RiskController
│   │   │   ├── event/             # TradeEventProducer/Consumer, ClearingConsumer
│   │   │   ├── config/            # KafkaTopicConfig (topic auto-creation)
│   │   └── controller/           # TestEventController (manual trade publish for testing)
│   └── market-data-gateway/       # Kafka → WebSocket bridge
│       ├── src/main/java/com/orderflow/market_data_gateway/
│       │   ├── kafka/             # MarketDataKafkaListener
│       │   └── config/            # WebSocketConfig (STOMP), KafkaErrorHandlingConfig
├── frontend/
│   ├── src/
│   │   ├── components/           # OrderEntryForm, OrderBook, CanvasOrderBook, DepthVisualizer, TradeList, TickerTape
│   │   └── services/             # websocketService.js (STOMP client)
```

Prerequisites

- Java 21 and Maven (or use the bundled `mvnw` in each service)
- A running Kafka broker on `localhost:9092` (no `docker-compose.yml` is included in the repo — provide your own broker, e.g. via Confluent's or Bitnami's Kafka Docker image, or a local install)
- Node.js (18+) and npm, for the frontend

Running locally

Start Kafka first (`localhost:9092`), then start each service in its own terminal:

```
# 1. Risk service – owns accounts, must be up before orders are submitted
cd backend/kafka-risk-service
./mvnw spring-boot:run           # http://localhost:8083

# 2. Matching engine
cd backend/matching-engine
./mvnw spring-boot:run           # http://localhost:8085

# 3. Market data gateway
cd backend/market-data-gateway
./mvnw spring-boot:run           # http://localhost:8084

# 4. Frontend
cd frontend
npm install
npm run dev                       # http://localhost:5173
```

Seed a test account before submitting orders (matching-engine's `RiskCheckClient` fails closed, so orders for unknown accounts are always rejected):

```
curl -X POST http://localhost:8083/api/accounts \
-H "Content-Type: application/json" \
-d '{"accountId":"ACC001","balance":100000}'
```

The frontend hardcodes `accountId: "ACC001"` and `symbol: "AXLR"` when submitting orders (see `App.jsx`), so use that account ID when seeding via the terminal UI.

API reference

Matching Engine — `http://localhost:8085`

`POST /api/orders` — submit an order for matching.

Request body:

```
{
  "accountId": "ACC001",
  "symbol": "AXLR",
  "side": "BUY",
  "type": "LIMIT",
  "price": 101.50,
  "quantity": 10
}
```

- `side`: `BUY` | `SELL`
- `type`: `LIMIT` | `MARKET` (price is ignored for `MARKET` orders by the matcher, but the field is still used to compute `orderValue` for the risk check — send a representative price for market orders)
- Response: `200 OK` with the generated `orderId` (string), or `400 Bad Request` with a rejection message if the risk check fails.

Kafka Risk Service — `http://localhost:8083`

Method	Path	Description
POST	<code>/api/accounts</code>	Create/update an account: <code>{"accountId": "...", "balance": 100000}</code>
GET	<code>/api/accounts</code>	List all accounts
GET	<code>/api/accounts/{accountId}</code>	Get one account (404 if not found)
GET	<code>/api/accounts/{accountId}/check?orderValue=1234.50</code>	Returns <code>{accountId, orderValue, sufficientFunds}</code> — used internally by the matching engine
POST	<code>/test/publish-sample-trade</code>	Publishes a synthetic <code>TradeExecutionEvent</code> to <code>trade.executions</code> for testing the Kafka → WebSocket pipeline without going through the matching engine

GET	/h2-console	H2 web console for inspecting the <code>riskdb</code> in-memory database
-----	-------------	--

Market Data Gateway — `http://localhost:8084`

No REST endpoints — it only exposes the WebSocket handshake at `/ws` . See below.

WebSocket / STOMP topics

Connect via SockJS to `http://localhost:8084/ws` , then subscribe over STOMP:

Topic	Payload	Published from
<code>/topic/trades</code>	<code>TradeExecutionEvent</code> JSON (<code>tradeId</code> , <code>symbol</code> , <code>buyOrderId</code> , <code>sellOrderId</code> , <code>price</code> , <code>quantity</code> , <code>executedAt</code>)	Kafka topic <code>trade.executions</code> , relayed 1:1
<code>/topic/orderbook</code>	<code>{ "bids": [{price, volume}, ...], "asks": [...] }</code> (top 10 levels per side)	Kafka topic <code>order.book.updates</code> , relayed 1:1

Kafka topics

Both topics are auto-created by `kafka-risk-service` on startup (`KafkaTopicConfig` , 3 partitions, replication factor 1):

- `trade.executions` — one message per fill, keyed by symbol. Producers: matching-engine (real fills), kafka-risk-service `/test/publish-sample-trade` (synthetic, for testing). Consumers: kafka-risk-service (`TradeEventConsumer` logs receipt, `ClearingConsumer` logs settlement), market-data-gateway (relays to WebSocket).
- `order.book.updates` — top-of-book snapshot emitted every 100ms by `DepthBroadcaster` , regardless of whether the book changed. Consumer: market-data-gateway.

Configuration

Each service reads its own `src/main/resources/application.properties` . Notable values:

- `matching-engine` : `server.port=8085` ; risk service URL is **hardcoded** in `RiskCheckClient` as `http://localhost:8083/api/accounts` (not externalized to properties).
- `kafka-risk-service` : `server.port=8083` ; H2 console enabled at `/h2-console` ; `spring.jpa.hibernate.ddl-auto=update` (schema auto-created from `Account` , no migrations).
- `market-data-gateway` : `server.port=8084` ; failed Kafka messages retry 3x with a 1s backoff (`KafkaErrorHandlingConfig`) before being skipped.
- `frontend` : WebSocket URL is hardcoded in `src/services/websocketService.js` as `http://localhost:8084/ws` ; REST order URL is hardcoded in `App.jsx` as `http://localhost:8085/api/orders` . There is no `.env` -based configuration — changing ports requires editing source.
- CORS: both `OrderController` and `RiskController` allow only `http://localhost:5173` .

Testing

- `matching-engine` includes `LatencyBenchmarkTest` , a `@SpringBootTest` that publishes 100,000 orders through the Disruptor and prints observed orders/sec. It measures publish-side throughput to the ring buffer, not end-to-end matching or Kafka publish latency.
- Standard `*ApplicationTests` context-load smoke tests exist in all three services.
- There is no integration test that exercises the full order → match → Kafka → WebSocket path across services.

Known limitations

- No persistence for orders/trades**: the order book and trade history live only in the matching engine's memory; a restart loses all open orders and book state.
- No multi-symbol routing logic**: the order book class supports arbitrary symbols, but nothing partitions or labels order books per symbol at the service level — the current wiring assumes a single instrument (`AXLR`) end-to-end (see hardcoded symbol in the frontend).
- Hardcoded service URLs**: cross-service URLs (risk-check, WebSocket, order submission) are hardcoded rather than externalized via configuration or service discovery, so changing ports/hosts means editing source in multiple places.
- No authentication/authorization**: all REST and WebSocket endpoints are open; `accountId` is client-supplied and not verified.
- No root Maven module**: the three backend services aren't wired together as Maven modules, so there's no single `mvn install` that builds all of them — each must be built/run independently.